DAA Assignment 2 Report
Triangulation and Guard Placement in a Polygon with Holes

24B1021
24B1037

April 3, 2026

Overview

The program reads a polygon with holes, converts it into a single merged boundary using bridge insertion, decomposes the merged boundary into monotone pieces, triangulates those pieces, and finally computes a constructive guard set by 3-coloring the triangulation graph.

1 Assignment Goal

The input is a polygonal gallery consisting of one outer boundary and zero or more interior holes. Let:

- n be the total number of input vertices across the outer boundary and all holes,
- h be the number of holes.

The solver must:

1. triangulate the walkable region,
2. compute a valid point-guard set,
3. print the triangles and guard coordinates.

The implementation is organized around two formulas:

$$\text{number of triangles} = n + 2h - 2$$

$$\text{guard upper bound} = \left\lfloor \frac{n+h}{3} \right\rfloor$$

The first formula is a correctness target for triangulation. The second formula gives the theoretical upper bound used by the guard stage.

2 Modular Components

Here is a list of all the files submitted and their function:

Component	Responsibility
`inc/graph.hpp`	Defines the half-edge data model: `Vertex`, `HalfEdge`, and `Face`.
`inc/parser.hpp` and `src/parser.cpp`	Read the assignment input, normalize orientation, and build circular half-edge cycles.
`inc/polygon_utils.hpp` and `src/polygon_utils.cpp`	Provide cycle traversal, total-vertex counting, and topmost-hole extraction.
`inc/hole_merger.hpp` and `src/hole_merger.cpp`	Merge holes into the outer boundary by a sweep-line bridge insertion strategy.
`inc/triangulation.hpp` and `src/triangulation.cpp`	Build the merged polygon walk, classify vertices, add diagonals, extract faces, and output triangles.
`inc/guards.hpp` and `src/guards.cpp`	Build the triangulation graph, 3-color it, and choose a guard set.
`src/main.cpp`	Orchestrates the full pipeline for every test case.
`visualize.py`	Converts the text output into a triangulated graph along with the guards.
`Makefile`	Builds the executable `build/art_gallery` with C++17.
`test.sh`	Submission script that builds and runs the solver on a given input file.

To run the program with a custom input just run:

```
./test.sh <input_file>
```

3 End-to-End Pipeline

The solver is easiest to understand as a sequence of seven stages. The first three stages keep the gallery in half-edge form. After holes are bridged, the triangulation code works on a merged polygon walk indexed by occurrences along that walk.

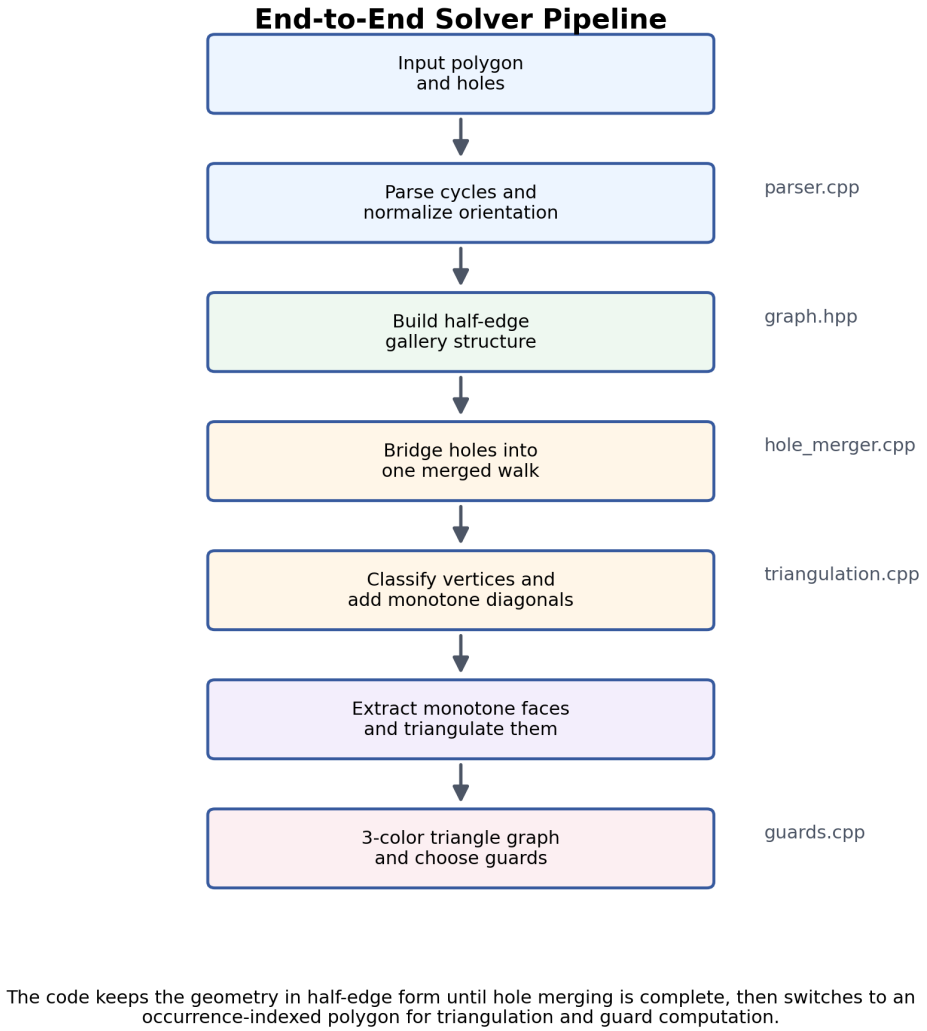

Figure 1: High-level structure of the current implementation.

The design choice that drives the whole solution is this:

> do the topological work first by removing holes through bridges, then run standard polygon triangulation ideas on the resulting merged boundary.

That keeps the difficult part localized inside the hole-merging phase and lets the later stages use well-known sweep-line and graph-coloring techniques.

4 Geometry and Topology Representation

4.1 Half-Edge Structures

The program does not build a full general-purpose DCEL. Instead, it uses a lightweight structure that is good enough for this task.

Field	Meaning in the implementation
`x`, `y` in `Vertex`	Geometric coordinates of one original input vertex.
`originatingEdge` in `Vertex`	One outgoing half-edge currently leaving that vertex. This pointer is important because it is updated when bridges are inserted.
`origin` in `HalfEdge`	The vertex at which the directed edge starts.
`nextEdge`, `prevEdge` in `HalfEdge`	The cyclic links that let the code walk a boundary or splice in new edges locally.
`twin` in `HalfEdge`	Used only for bridges. The pair `bridge` and `bridgeTwin` represent the two directions of the inserted connection.
`boundaryFace` in `HalfEdge`	The face to which the half-edge belongs. The code keeps this simple because the assignment only needs one gallery face.
`boundaryEdge` in `Face`	One representative edge on the outer boundary before merging, and one representative edge on the merged walk after merging.
`InnerComponents` in `Face`	One representative edge for each hole cycle before merging.

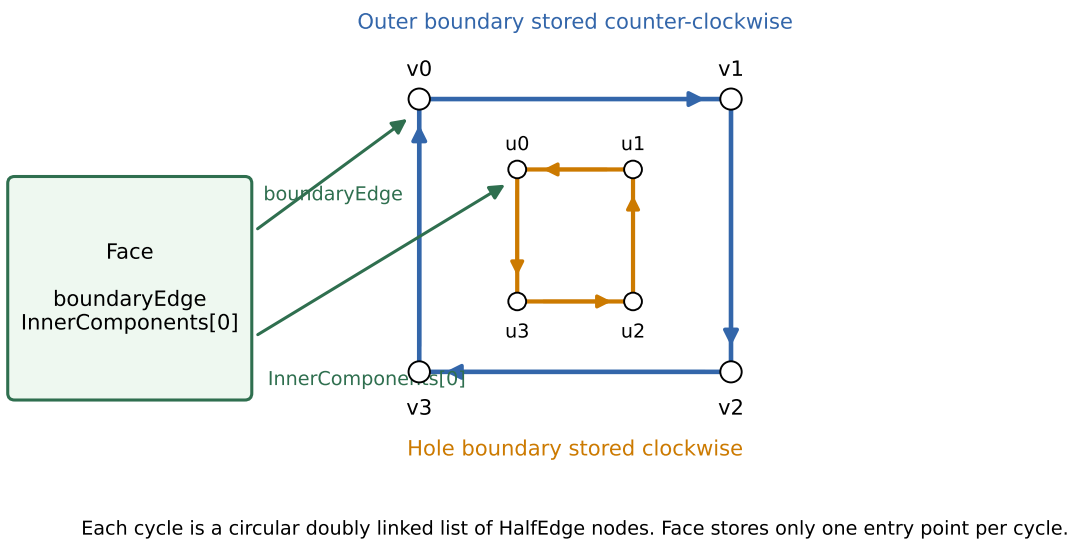

Figure 2: How the gallery is stored before hole merging. The outer boundary and each hole are separate circular half-edge cycles, and the face stores only one entry point into each cycle.

This representation is a strong fit for the problem because bridge insertion is purely local. When a bridge is added, the code only rewires a few pointers around the source and target vertices. It never rebuilds the gallery from scratch.

4.2 Orientation Invariant

The parser immediately normalizes orientation:

- the outer boundary is stored counter-clockwise,
- every hole is stored clockwise.

This invariant matters for three reasons:

- local cross-product tests become consistent,
- helper and bridge directions are interpreted correctly,
- the interior stays on the expected side of the walk.

4.3 Parsing Logic

For each cycle, `src/parser.cpp` does the following:

1. read all coordinates into a temporary vector,
2. compute the signed area,
3. reverse the order if the orientation is opposite to the required convention,
4. allocate one `Vertex` and one `HalfEdge` per coordinate,
5. connect the half-edges into a circular doubly linked list.

After parsing is complete, `src/polygon_utils.cpp` supplies the shared traversal helpers used later by hole merging and guard computation:

- `addVertices` walks a cycle and collects its vertices,
- `getTopmostVertices` extracts the topmost vertex of each hole,
- `getTotalVertexCount` counts the original input vertices before topology changes happen.

5 Hole Merging

5.1 Why Hole Merging Comes First

The triangulation algorithm expects a single polygonal walk. A gallery with holes is therefore converted into one merged boundary before triangulation begins.

The implementation in `src/hole_merger.cpp` performs a top-down sweep over all vertices. The key event is the moment when the sweep reaches the topmost vertex of a hole. At that moment, the hole is connected to the nearest valid wall on its left.

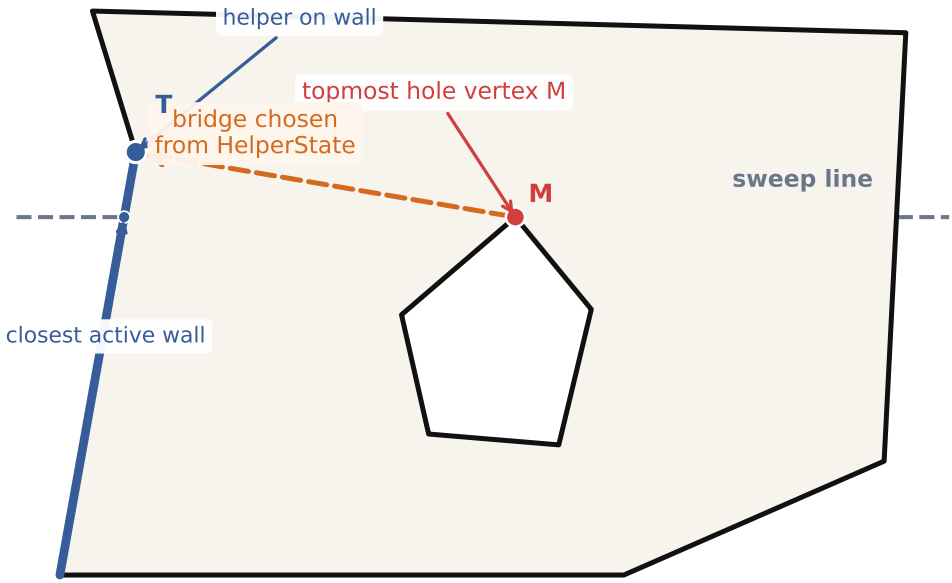

Figure 3: Bridge search during the sweep. The topmost hole vertex M queries the closest active wall on its left, and the wall's current `HelperState` determines the bridge target.

5.2 Three Tracking Structures Drive the Merge

The Hole Merger maintains three synchronized structures because the sweep has to distinguish geometry, local topology, and boundary-component ownership:

1. **Active-edge map.** A balanced map from active wall edges to full `HelperState` records. The key order comes from sweep-line intersection on the current horizontal ray.
2. **Outgoing-angle index.** A per-vertex map of outgoing half-edges ordered by direction angle is used to choose the exact splice location after a target vertex has been selected.
3. **Component map.** A table from each original `Vertex*` to its source component records whether that vertex belongs to the outer boundary or to one specific hole cycle.

The three structures answer three different questions:

- which wall is immediately to the left of the current sweep event,
- which helper information on that wall should survive same-level updates,
- where exactly the bridge should be spliced around the chosen target vertex.

5.3 What HelperState Stores

The `HelperState` structure is what allows proper helper vertex tracking while merging.

Field	Purpose
`helper`	The currently preferred helper vertex on that active wall.
`lockedAtY`	The sweep-line height at which a hole-top event locked this helper state.
`lockedComponentId`	The boundary component that is allowed to keep updating the helper while the same-level lock is active.
`lockSameLevel`	Whether same-y overwrites are currently restricted.
`minLockedHelper` and `maxLockedHelper`	The leftmost and rightmost helper candidates seen at the locked y-level for the same component. These let the code choose a better bridge target later.

Table 1: State stored per active wall in the updated hole merger.

The helper-related support functions divide the update logic cleanly:

- `makeUnlockedHelper` and `setUnlockedHelper` create ordinary unlocked helper states,
- `setHoleTopHelper` locks an active wall to a hole-top event,
- `canOverwriteWithOrdinary` decides whether an ordinary sweep vertex may replace a locked helper,
- `applyOrdinaryHelperUpdate` updates the helper state while preserving same-level extrema,
- `getBridgeTarget` chooses which locked helper endpoint should actually receive the bridge.

Two small implementation details matter here. First, the component table is built once before the sweep so that every overwrite decision can check component ownership in constant time. Second, the outgoing-angle index uses an `OutgoingKey` made from the outgoing direction angle and a unique edge id, so multiple outgoing edges with the same angle can still coexist in the map without collisions.

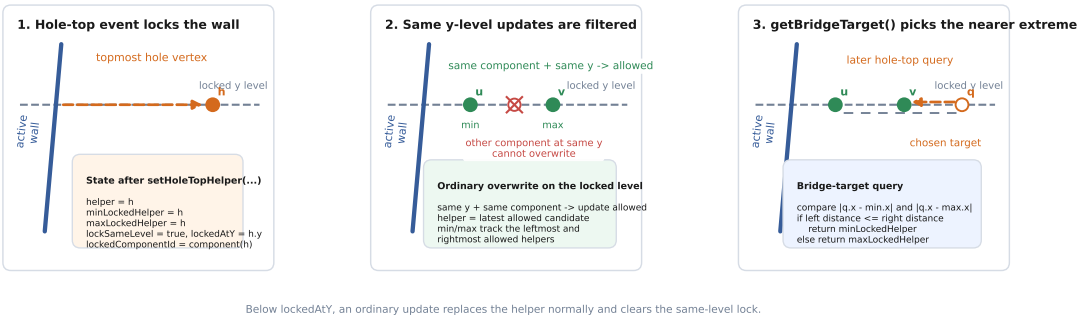

Figure 4: Same-level helper locking in the updated merge logic. When a hole-top event locks an active wall, only the same component may keep refining helper information at that y-level, and `getBridgeTarget()` can later choose the closer of the tracked leftmost/rightmost locked helpers.

5.4 Why the Same-Level Lock Exists

The lock solves a subtle sweep-order problem. Suppose a hole-top event occurs at height y_0, and several vertices on the left wall also lie on that same height. The code wants all same-level vertices from the *same* boundary component to be treated as one logical helper region, but it does not want another component at the same height to overwrite that state before the bridge choice is finalized.

That is exactly what the lock enforces:

- if a candidate vertex is strictly below the locked height, the helper can be replaced normally,
- if a candidate vertex is at the same height, it may refine the helper only if it comes from the same component,
- while that happens, the code remembers both the minimum-x and maximum-x helper on the locked level.

The implementation compares heights using a small tolerance `kHelperLockEps = 1e-9`. In effect, the code treats vertices on the same geometric level as belonging to one locked band, then filters that band by component id.

When a later hole top queries the same wall, `getBridgeTarget()` compares the query vertex against those two stored extremes and chooses the closer one in x.

5.5 Actual Merge Sequence in the Code

The current code follows this sequence:

1. Use `getTopmostVertices` to select one bridge source per hole.
2. Sort all vertices from top to bottom, breaking equal-y ties by smaller x.
3. Initialize the outgoing-angle index with the outer cycle and every hole cycle.
4. Build `vertexComponentIds` so every original vertex knows which boundary component it belongs to.
5. Sweep through the sorted vertices.
6. Before modifying the sweep state at a vertex, remember the current closest left wall as `preLeftWall`.
7. If the current vertex is the next hole-top vertex, use the stored helper state of `preLeftWall` to choose the actual target vertex on that wall.
8. Use the outgoing-angle index to choose a valid outgoing half-edge at the target vertex.
9. Insert `bridge` and `bridgeTwin`, then rewire only the nearby `nextEdge`/`prevEdge` links.
10. Update active edges for the current vertex; newly active edges receive ordinary unlocked helper states.
11. Recompute the closest left wall after those updates.
12. If a hole top was processed, lock the helper state on the relevant active wall using `setHoleTopHelper`.
13. If no hole top was processed, first test whether an ordinary overwrite is allowed.
14. Only then apply the ordinary helper update.
15. If the pre-bridge left wall is still active after the bridge update, relock it so the hole-top information is not lost when the active-edge neighborhood changes.

This logic appears in the following key functions:

- `getRayIntersectionX`: computes the x-coordinate where an edge meets the sweep line,
- `findClosestWallToLeft`: binary-searches the active-edge map,
- `addCycleVerticesToComponentMap`: records the source component of every original vertex,
- `getBridgeTarget`: chooses a bridge endpoint from the locked helper state,
- `findValidSplicePoint`: chooses the correct outgoing edge at the target vertex,
- `buildBridge`: allocates the bridge pair and rewires local topology.

5.6 Why the Target Vertex Needs Angular Checks

Once one bridge has already been inserted, a later bridge may hit a vertex that now has multiple outgoing edges. If the new bridge is inserted against the wrong outgoing edge, the merged walk becomes topologically incorrect even if the bridge endpoints themselves were geometrically sensible.

That is why the code uses:

- bridge direction angle,
- outgoing half-edge angle,
- a sector-membership test.

Together, these checks make the splice robust against earlier modifications around the target vertex. The ordered outgoing map stores keys of the form `(angle, edgeId)`, and `findValidSplicePoint()` probes that map with `upper_bound`. This lets the code locate the outgoing edge immediately preceding the desired bridge direction and then verify that the bridge lies inside the correct local sector.

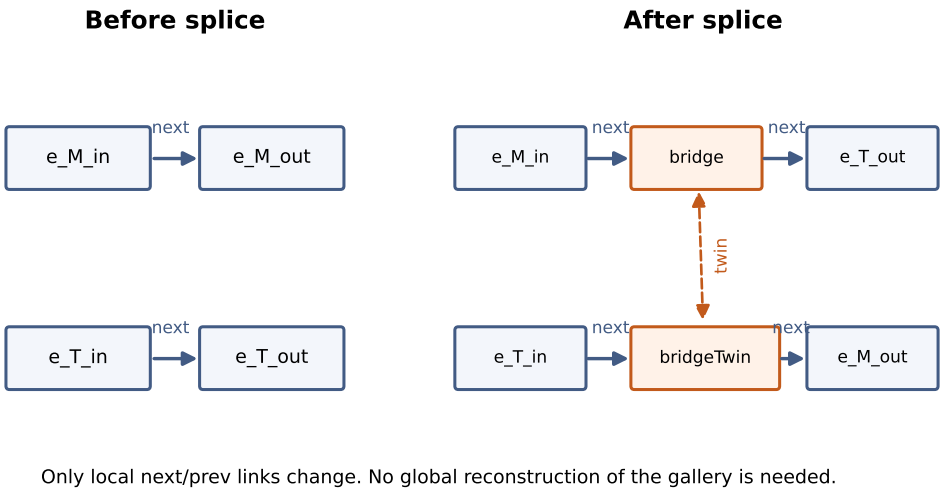

Figure 5: Local pointer changes performed when a bridge is inserted. Only the neighborhood of the source and target vertices is modified.

5.7 State After Merging

After all holes are processed:

- the outer boundary and holes are no longer separate cycles,
- `gallery->boundaryEdge` reaches one merged boundary walk,
- some original vertices can now appear more than once when that merged walk is traversed.

That last detail is crucial. The triangulation stage does not treat the merged boundary as a set of unique coordinates. It treats it as a sequence of *occurrences* along a walk.

6 Triangulation

6.1 Merged Walk as an Occurrence-Indexed Polygon

The program in `src/triangulation.cpp` first traverses the merged boundary and copies each visited vertex into a `PolygonVertex` record:

- the coordinates are copied directly,
- the original `Vertex*` is preserved,
- the position in the traversal becomes the occurrence index.

This is a deliberate design choice. The code no longer perturbs coordinates to separate repeated bridge endpoints. Instead, it distinguishes occurrences combinatorially by their position in the merged walk.

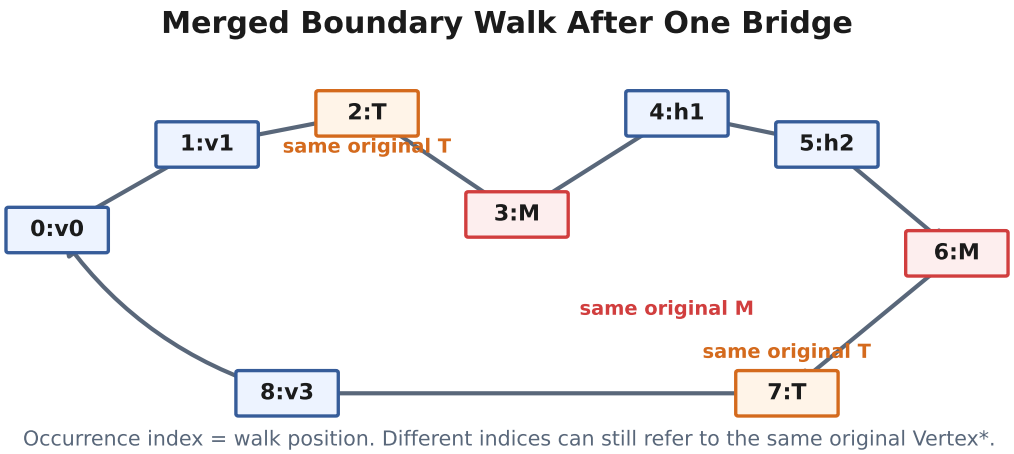

Figure 6: Why occurrence indices matter after bridging. The same original geometric vertex can appear more than once in the merged walk.

This is also why the triangulation result stores both:

- `triangleIndices`: triangles in occurrence space,
- `occurrenceVertices`: a map back from occurrence indices to original `Vertex*` pointers.

6.2 Vertex Classification

Every occurrence is classified as one of the six standard sweep-line categories:

- `START`,
- `END`,
- `SPLIT`,
- `MERGE`,
- `REGULAR_LEFT`,
- `REGULAR_RIGHT`.

The classification uses:

- relative sweep order of the previous and next vertices,
- the sign of the orientation test around the current vertex.

In exact code terms, the six outcomes are:

- `START`: `prevBelow` and `nextBelow` are both true, and `ccw(prev, curr, next) > eps`.
- `SPLIT`: `prevBelow` and `nextBelow` are both true, and `ccw(prev, curr, next) <= eps`.
- `END`: `prevBelow` and `nextBelow` are both false, and `ccw(prev, curr, next) > eps`.
- `MERGE`: `prevBelow` and `nextBelow` are both false, and `ccw(prev, curr, next) <= eps`.
- `REGULAR_LEFT`: the vertex is in the mixed case and `isAbove(prev, curr)` is true.
- `REGULAR_RIGHT`: the vertex is in the mixed case and `isAbove(prev, curr)` is false.

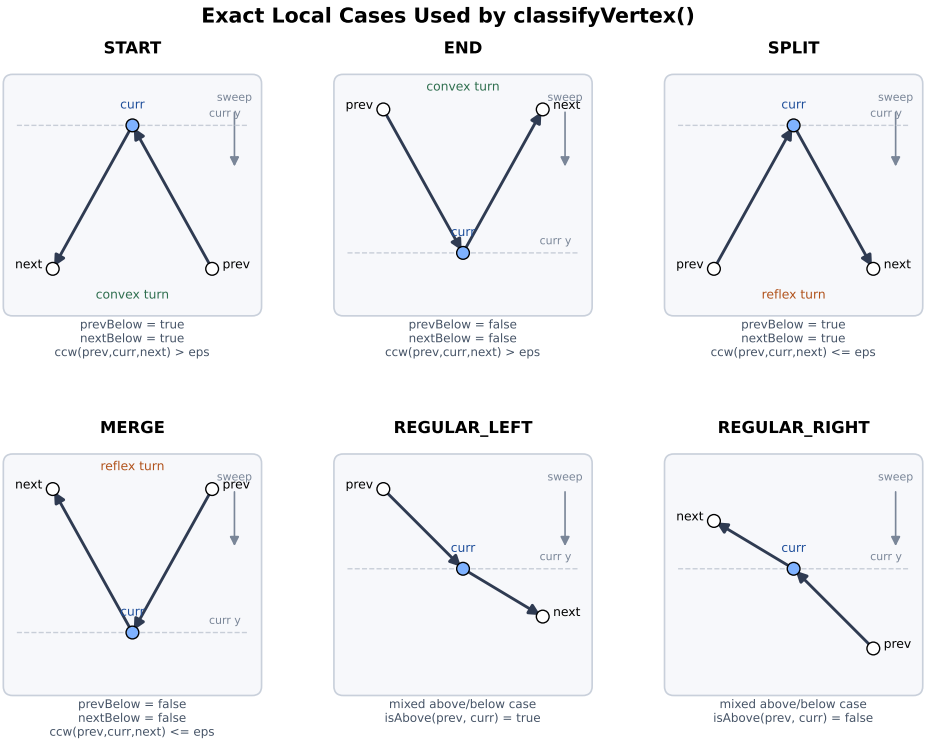

Figure 7: Representative local configurations for the exact predicates used by `classifyVertex`. The panels follow the counter-clockwise walk `prev -> curr -> next`, and the meaning of above/below matches the same sweep-order tie-breaks used in the code.

6.3 Monotone Decomposition

The function `makeMonotone` builds a temporary sweep boundary and then processes occurrence vertices from top to bottom. Its active-edge structure stores helper occurrence indices rather than raw geometry objects. The diagonal rules follow the standard monotone-decomposition sweep:

- start vertices insert their outgoing edge,
- end vertices may connect to a merge helper on the incoming edge,
- split vertices connect to the helper of the closest left wall and then insert the outgoing edge,
- merge vertices may need one diagonal to the incoming-edge helper and another to the closest left-wall helper,
- regular vertices either update the left-wall helper or replace the incoming edge by the outgoing edge, depending on which chain they lie on.

The current implementation also has several practical filters in `addDiagonal`. These are necessary because repeated bridge endpoints can create multiple occurrences with identical coordinates.

- A vertex is never connected to itself.
- Adjacent polygon vertices are never treated as diagonals.
- Two occurrences with the same coordinates are not connected directly.

- Existing polygon edges are not re-added as diagonals.
- Duplicate undirected diagonals are ignored.

6.4 Recovering Faces Without Building Another DCEL

After monotone diagonals are known, the program still needs the actual monotone faces. The code solves that problem in a distinctive way:

1. place the occurrence indices on a synthetic circle,
2. add the polygon boundary edges and the diagonals into one undirected graph,
3. sort each adjacency list by angular order on that circle,
4. walk directed edges while always turning to the previous angular neighbor,
5. keep only the positive-area cycles.

Two details matter for correctness here:

- the embedding is the same one built by the code, namely occurrence i placed at angle $2\pi i/m$,
- the diagonals must stay noncrossing when drawn on the circular placement, so the resulting graph can be traced face by face.

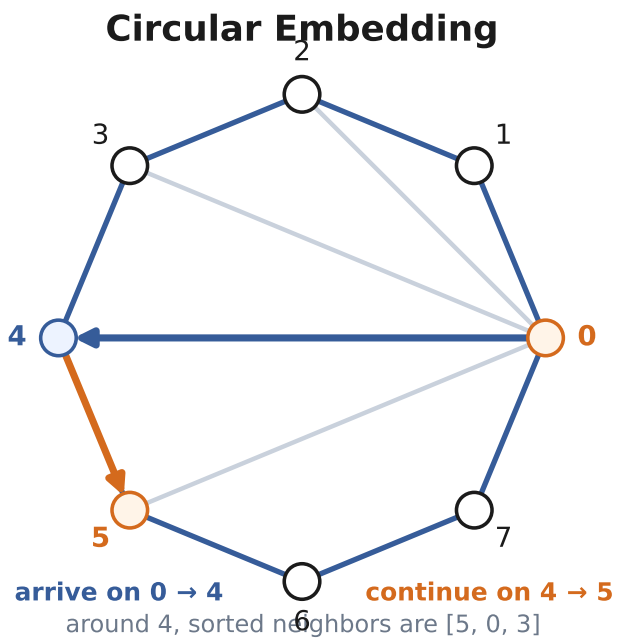

Figure 8: Circular embedding used in `extractMonotoneFaces`. The occurrence indices are placed on the same counter-clockwise circle used by the code, the diagonals remain noncrossing, and the highlighted local step shows the actual traversal rule: arriving along `0 -> 4` makes the walk continue along `4 -> 5` because 5 is the previous angular neighbor of 0 in the sorted rotation order around 4.

This avoids introducing another heavy topology structure while still recovering faces reliably from the diagonal graph.

6.5 Triangulating Each Monotone Face

For every extracted monotone face, `triangulateMonotoneFace` runs the standard stack-based algorithm:

1. identify the top and bottom vertices,
2. mark the left chain and right chain,
3. process vertices in sweep order,
4. pop from the stack while the local chain test says a diagonal is valid,
5. add the final diagonals to close the fan at the last vertex.

Then the code extracts faces one more time. Any face with exactly three non-degenerate vertices is accepted by `appendTriangleIfValid`. This function also normalizes orientation so the printed triangle order is consistent.

7 Guard Computation

7.1 Bound and Graph Construction

The guard stage uses the original input counts recorded *before* hole merging:

$$\texttt{theoreticalBound} = \left\lfloor \frac{n+h}{3} \right\rfloor$$

This value is stored in `GuardSolution`, but the solver does not blindly print that bound. Instead, it tries to construct a smaller valid set when possible.

The graph built by `src/guards.cpp` has:

- one graph vertex per occurrence in the triangulation,
- one edge between every pair of occurrence indices that share a triangle.

The boundary edges and inserted diagonals form a graph that can be drawn on the same circle without any edge crossings, with every occurrence index staying on the outer boundary.

7.2 3-Coloring

The function `colorGraph` colors the graph in two passes:

1. repeatedly remove a vertex of degree at most two, recording the removal order,
2. traverse that order backward and color each vertex with the first available color.

This works because, in a correct noncrossing triangulation drawn on the circular embedding, there is always a remaining occurrence with only two unprocessed neighbors, so the elimination step can keep going.

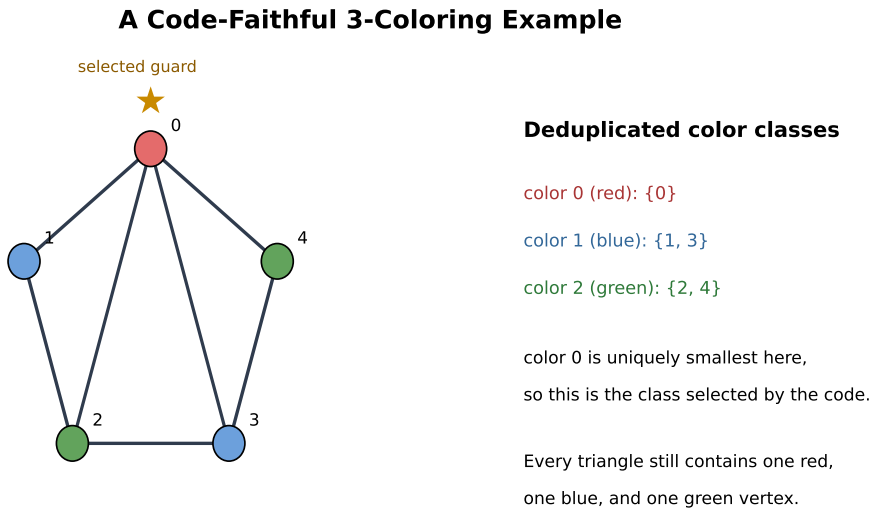

Figure 9: A code-faithful coloring example. The red class is uniquely smallest after deduplicating original `Vertex*` pointers, so it is the one selected by the current implementation.

After coloring, the code:

- gathers original `Vertex*` pointers by color class,
- deduplicates them with ordered sets,
- selects the smallest color class as the guard set.

This is why the actual guard count printed by the program can be smaller than the theoretical bound.

If two color classes tie after deduplication, the current code keeps the earliest minimum it encounters because `bestColor` is updated only on a strict improvement.

7.3 Fallback Behavior

If the coloring step somehow fails, the code falls back to the first distinct original vertices it can collect, up to the theoretical bound. This fallback is conservative, but it keeps the solver from producing an empty answer in the presence of an unexpected coloring failure.

8 Complexity Analysis and Engineering Notes

8.1 Notation Used for the Analysis

Let:

- n be the total number of original input vertices,
- h be the number of holes,
- m be the number of vertex *occurrences* on the merged walk after hole bridging.

In this implementation, each hole bridge causes two bridge endpoints to reappear in the merged boundary traversal, so:

$$m = n + 2h$$

This is consistent with the triangle formula for a simple polygon obtained after bridging:

$$m - 2 = n + 2h - 2$$

So the later triangulation and guard stages are analyzed in terms of m, and then converted back to (n, h) at the end.

8.2 Parsing and Utility Routines

Parsing one test case.

- Reading all coordinates takes $\mathcal{O}(n)$ time.
- Computing signed area for each cycle is linear in that cycle size, so the total is still $\mathcal{O}(n)$.
- Allocating one `Vertex` and one `HalfEdge` per input vertex is $\mathcal{O}(n)$.
- Linking each cycle into a circular doubly linked list is $\mathcal{O}(n)$.

So the full parsing stage costs $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ space.

Utility routines in `src/polygon_utils.cpp`.

- `addVertices`: $\mathcal{O}(k)$ for a cycle of length k.
- `getMergedPolygonVertices`: $\mathcal{O}(m)$ after hole merging.
- `getTopmostVertices`: scans every hole cycle once, so $\mathcal{O}(n)$ total before merging.
- `getTotalVertexCount`: scans every cycle once, so $\mathcal{O}(n)$.
- `getVerticesSorted`: collect all vertices in $\mathcal{O}(n)$ and sort them in $\mathcal{O}(n \log n)$, giving $\mathcal{O}(n \log n)$ overall.

8.3 Hole Merging Complexity

The updated hole-merging phase has five distinct costs.

1. Topmost-hole extraction.

`getTopmostVertices` scans all hole cycles once, so this part is $\mathcal{O}(n)$.

2. Sweep-order preparation.

All original vertices are collected and sorted by sweep priority, costing $\mathcal{O}(n \log n)$.

3. Index initialization before the sweep.

Two auxiliary structures are prepared before the first sweep event:

- the outgoing-angle index inserts one initial outgoing edge per original vertex, costing $\mathcal{O}(n)$ total,

- `addCycleVerticesToComponentMap` records one component id per original vertex, also costing $\mathcal{O}(n)$ total.

4. Main sweep over vertices.

The sweep visits every original vertex once. For each event it performs only a constant number of:

- active-edge map lookups,
- active-edge insertions and deletions,
- `upper_bound` queries,
- helper-state updates.

The helper-state routines such as `setHoleTopHelper`, `canOverwriteWithOrdinary`, and `applyOrdinaryHelperUpdate` are constant-time because they only read or write a few fields, including the tracked minimum and maximum same-level helpers. The active-edge map is still a balanced `std::map`, so the dominant per-event cost is $\mathcal{O}(\log n)$. Therefore the full sweep contributes $\mathcal{O}(n \log n)$.

5. Hole-top bridge events.

When a hole topmost vertex is encountered, the extra work is:

- one active-edge lookup for the pre-bridge left wall,
- one constant-time `getBridgeTarget` decision from the stored helper extrema,
- one search in the outgoing-angle index, costing $\mathcal{O}(\log d(v))$ where $d(v)$ is the number of outgoing edges currently stored for the chosen target vertex,
- constant-time pointer rewiring in `buildBridge`,
- two outgoing-edge insertions,
- one extra active-edge lookup to preserve the pre-bridge wall lock if that wall remains active.

Across all holes this remains within $\mathcal{O}(h \log n)$ worst-case time, which is still dominated by the global sort and the sweep itself.

Hole-merging total.

$$\text{Time} = \mathcal{O}(n \log n)\,, \qquad \text{Space} = \mathcal{O}(n + h)$$

The extra space is due to:

- the sorted vertex list,
- the active-edge map, now storing one `HelperState` per active wall,
- the outgoing-angle index,
- the component-id map,
- the bridge half-edges added for each hole.

8.4 Triangulation Complexity

The triangulation stage is best analyzed in terms of the merged-walk length $m = n + 2h$.

1. Merged polygon construction.

- `buildMergedPolygon` traverses the merged boundary once: $\mathcal{O}(m)$ time.
- Storing occurrence vertices also takes $\mathcal{O}(m)$ space.
- Orientation correction by a polygon-area computation is another $\mathcal{O}(m)$.

2. Vertex classification.

Each occurrence is classified using only neighboring occurrences and a constant amount of arithmetic, so the total is $\mathcal{O}(m)$ time and $\mathcal{O}(m)$ space for the category array.

3. Monotone decomposition.

Inside `makeMonotone`:

- sorting occurrence indices by sweep order costs $\mathcal{O}(m \log m)$,
- building the temporary sweep boundary costs $\mathcal{O}(m)$,
- the active-edge sweep performs a constant number of balanced-map operations per occurrence, costing $\mathcal{O}(m \log m)$ overall,
- diagonal deduplication uses a set, so insert/check operations are $\mathcal{O}(1)$ average per diagonal.

So `makeMonotone` runs in $\mathcal{O}(m \log m)$ time and uses $\mathcal{O}(m)$ extra space.

4. Face extraction after adding diagonals.

The function `extractMonotoneFaces` does:

- build the synthetic circular embedding: $\mathcal{O}(m)$,
- build adjacency from polygon edges and diagonals: $\mathcal{O}(m)$ because the subdivision is planar and has only linear-size edge sets,
- sort each adjacency list by angle: worst-case $\mathcal{O}(m \log m)$ over all vertices,
- walk each directed edge once to recover faces: $\mathcal{O}(m)$.

Therefore one call to `extractMonotoneFaces` is $\mathcal{O}(m \log m)$ time and $\mathcal{O}(m)$ space in the worst case.

5. Triangulating monotone faces.

For a monotone face of size s, `triangulateMonotoneFace` performs:

- face simplification in $\mathcal{O}(s)$,
- top/bottom detection in $\mathcal{O}(s)$,
- sorting that face's vertices by sweep order in $\mathcal{O}(s \log s)$,
- stack-based triangulation in $\mathcal{O}(s)$.

Across all monotone faces, the total size of all face boundaries is linear in the subdivision size, so the aggregate worst-case cost is:

$$\mathcal{O}(m \log m)$$

6. Final triangular-face extraction and output assembly.

After all triangulation diagonals are added, the code calls `extractMonotoneFaces` one more time and then runs `appendTriangleIfValid` on every face. That contributes another $\mathcal{O}(m \log m)$ worst-case time and $\mathcal{O}(m)$ space.

Triangulation total.

$$\text{Time} = \mathcal{O}(m \log m)\,, \qquad \text{Space} = \mathcal{O}(m)$$

Substituting $m = n + 2h$, this becomes:

$$\text{Time} = \mathcal{O}((n + 2h)\log(n + 2h))\,, \qquad \text{Space} = \mathcal{O}(n + 2h)$$

8.5 Guard Computation Complexity

Let t be the number of output triangles. Since $t = m - 2$, we have $t = \Theta(m)$.

1. Building the triangulation adjacency graph.

Each triangle contributes exactly three undirected edges, so `buildAdjacency` runs in $\mathcal{O}(t) = \mathcal{O}(m)$ time and uses $\mathcal{O}(m)$ space.

2. Reverse-degeneracy coloring.

The queue-based elimination phase touches each graph vertex once and each adjacency relation a constant number of times, so it is $\mathcal{O}(m)$. The reverse coloring pass is also $\mathcal{O}(m)$.

Thus the coloring procedure itself is:

$$\text{Time} = \mathcal{O}(m)\,, \qquad \text{Space} = \mathcal{O}(m)$$

3. Deduplicating original vertices by color class.

After coloring, the code inserts original `Vertex*` pointers into three `std::set` containers. Since ordered-set insertion is $\mathcal{O}(\log m)$ and there are $\mathcal{O}(m)$ occurrence insertions, this step is $\mathcal{O}(m \log m)$ in the worst case.

4. Fallback path.

If coloring fails, the fallback builds one `std::set<Vertex*>` from all occurrence vertices, which is also $\mathcal{O}(m \log m)$ worst-case.

Guard-stage total.

Because of the ordered-set deduplication used in the current code, the overall guard stage is:

$$\text{Time} = \mathcal{O}(m \log m)\,, \qquad \text{Space} = \mathcal{O}(m)$$

8.6 Printing Complexity

- Printing triangles costs $\mathcal{O}(t) = \mathcal{O}(m)$.
- Printing guards costs $\mathcal{O}(g)$ where g is the number of selected guards, and trivially $g \leq m$.

So the output stage is linear in the size of the produced answer.

8.7 Overall Complexity of the Full Algorithm

Putting all stages together:

- parsing and utility preparation: $\mathcal{O}(n \log n)$ worst-case because of sorting,
- hole merging: $\mathcal{O}(n \log n)$,
- triangulation on the merged walk: $\mathcal{O}(m \log m)$,
- guard computation: $\mathcal{O}(m \log m)$,
- printing: $\mathcal{O}(m)$.

Since $m = n + 2h$, the full solver runs in:

$$\text{Time} = \mathcal{O}((n + 2h) \log(n + 2h))$$

and uses:

$$\text{Space} = \mathcal{O}(n + 2h)$$

If the standard assumption is used that $h \leq n$, then this simplifies to:

$$\text{Time} = \mathcal{O}(n \log n)\,, \qquad \text{Space} = \mathcal{O}(n)$$

Some important details:

- `src/main.cpp` records `totalVertices` and `holeCount` before merging so the theoretical guard bound uses the original gallery, not the modified merged walk.
- `mergeHoles` updates `originatingEdge` at bridge endpoints so later traversal naturally follows the merged topology.
- The updated hole merger keeps component-aware helper locks, so same-level sweep events from another boundary component cannot accidentally overwrite the helper information that a hole-top bridge decision still depends on.
- The triangulation stage uses occurrence indices instead of trying to geometrically separate repeated bridge endpoints.
- The guard stage prints the actual number of selected guards, not the theoretical upper bound.

9 Worked Examples

The following examples were produced by running the current executable on representative gallery geometries. Each example is described directly here, without depending on external testcase names.

Example	n	h	Expected triangles	Actual guards
Convex polygon	4	0	2	1
Concave simple polygon	8	0	6	2
One-hole gallery	8	1	8	3
Two-hole gallery	20	2	22	5

Table 2: Summary of the example runs discussed below.

9.1 Example 1: Convex Polygon

Geometry:

- outer boundary: $(0,0)$, $(6,0)$, $(6,6)$, $(0,6)$,
- holes: none.

$$n = 4, \qquad h = 0, \qquad n + 2h - 2 = 2, \qquad \left\lfloor \frac{n+h}{3} \right\rfloor = 1$$

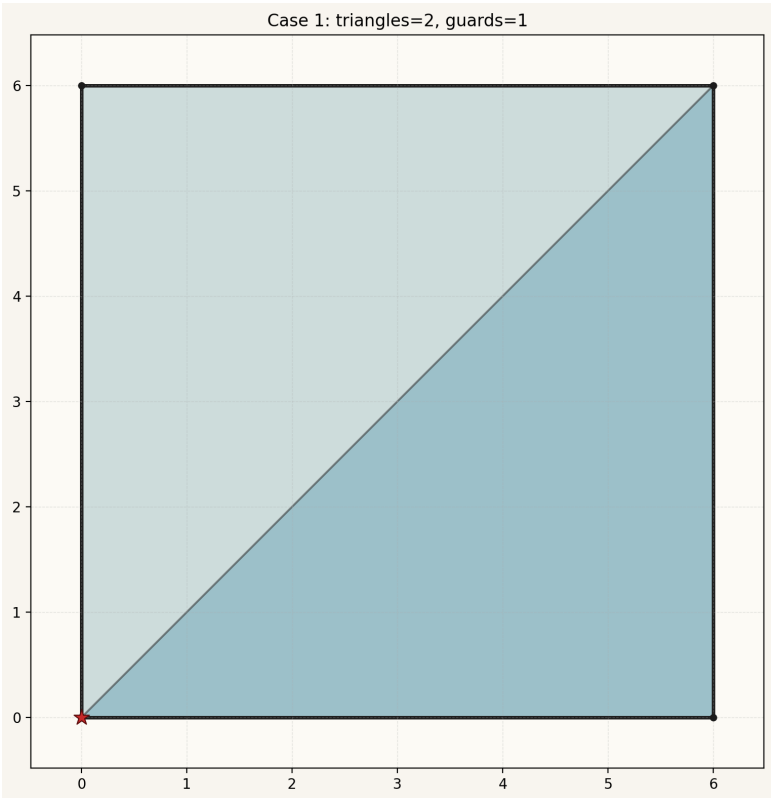

Figure 10: Convex gallery: the solver adds one diagonal and one guard is enough.

Actual output:

```
2
0 0 6 0 6 6
0 0 6 6 0 6
1
0 0
```

This example is the clean baseline for the entire pipeline. There are no holes and no reflex vertices, so the output checks that the parser, triangulation printer, and guard printer all behave correctly in the simplest possible case.

9.2 Example 2: Concave Simple Polygon

Geometry:

- outer boundary: $(0,0)$, $(8,0)$, $(8,2)$, $(5,2)$, $(5,5)$, $(8,5)$, $(8,7)$, $(0,7)$,
- holes: none.

$$n = 8, \qquad h = 0, \qquad n + 2h - 2 = 6, \qquad \left\lfloor \frac{n+h}{3} \right\rfloor = 2$$

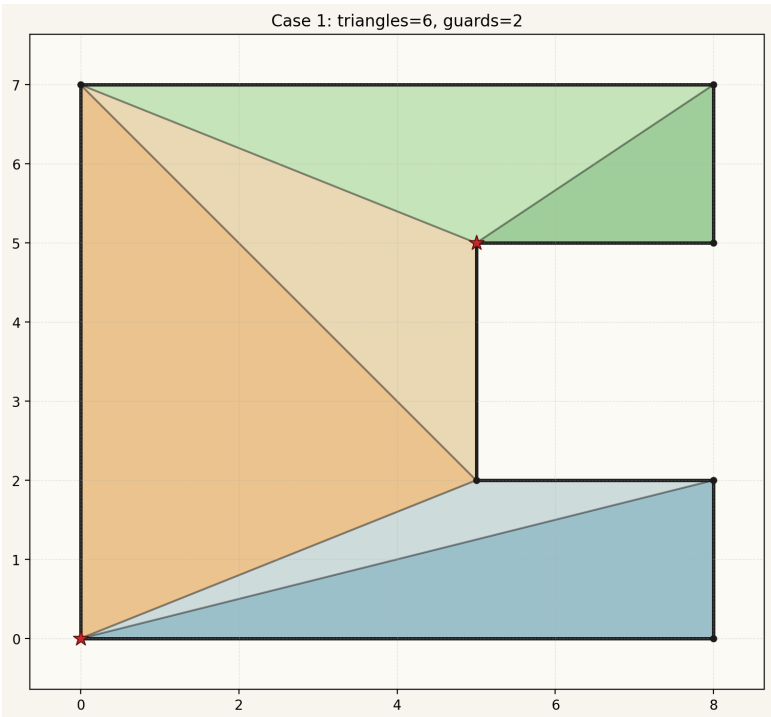

Figure 11: Concave simple polygon: monotone decomposition and graph coloring are both active.

Actual output:

```
6
0 0 8 0 8 2
0 0 8 2 5 2
0 0 5 2 0 7
```

```
5 2 5 5 0 7
5 5 8 5 8 7
5 5 8 7 0 7
2
0 0
5 5
```

This example is the first one that exercises reflex geometry. The code has to classify non-convex vertices correctly, add the right diagonals, and still end with exactly six triangles.

9.3 Example 3: One-Hole Gallery

Geometry:

- outer boundary: $(0,0)$, $(12,0)$, $(12,12)$, $(0,12)$,
- one hole: $(3,3)$, $(3,8)$, $(8,8)$, $(8,3)$.

$$n = 8, \qquad h = 1, \qquad n + 2h - 2 = 8, \qquad \left\lfloor \frac{n+h}{3} \right\rfloor = 3$$

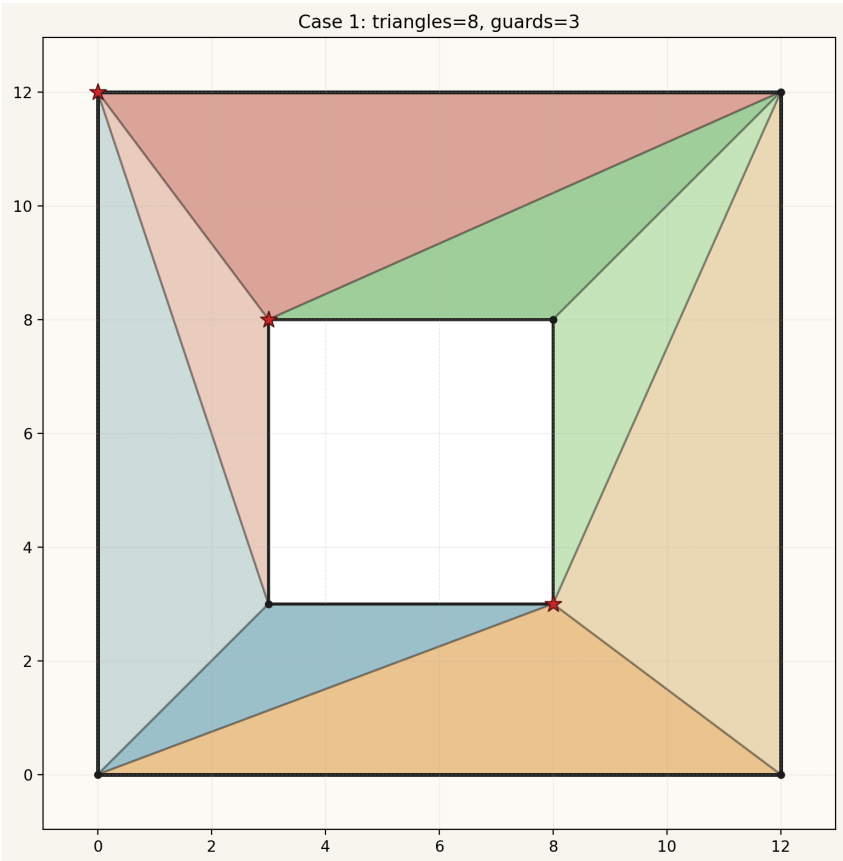

Figure 12: One-hole gallery: one bridge event creates a merged walk and the solver triangulates the resulting region into eight triangles.

Actual output:

```
8
0 0 8 3 3 3
0 0 3 3 0 12
0 0 12 0 8 3
12 0 12 12 8 3
12 12 3 8 8 8
12 12 8 8 8 3
12 12 0 12 3 8
3 3 3 8 0 12
3
0 12
3 8
8 3
```

This example shows why the bridge code matters. A correct run must:

- identify the topmost vertex of the hole,
- connect it to a valid target on the outer boundary,
- traverse the merged walk without losing repeated bridge endpoints,
- preserve the helper state correctly while the bridge changes the local sweep neighborhood,
- still produce exactly eight triangles.

9.4 Example 4: Irregular Gallery with Two Holes

Geometry:

- outer boundary: $(0,0)$, $(10,0)$, $(13,2)$, $(14,6)$, $(12,10)$, $(8,12)$, $(4,11)$, $(1,9)$, $(-1,6)$, $(-2,3)$, $(-1,1)$,
- hole 1: $(2,2)$, $(4,1.5)$, $(5.5,3.5)$, $(4.5,5.5)$, $(2,5)$,
- hole 2: $(8,6)$, $(10,5.5)$, $(10.5,8)$, $(8.5,8.5)$.

$$n = 20, \qquad h = 2, \qquad n + 2h - 2 = 22, \qquad \left\lfloor \frac{n+h}{3} \right\rfloor = 7$$

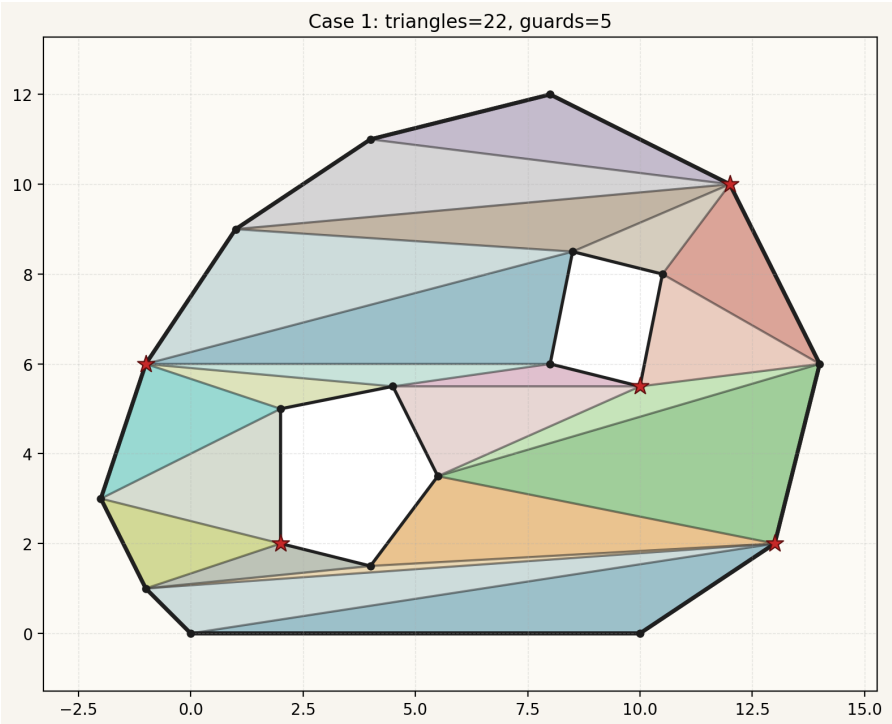

Figure 13: Irregular multi-hole gallery: the strongest example in the report.

Actual output:

```
22
0 0 10 0 13 2
0 0 13 2 -1 1
13 2 5.5 3.5 4 1.5
13 2 4 1.5 -1 1
13 2 14 6 5.5 3.5
14 6 10 5.5 5.5 3.5
14 6 12 10 10.5 8
14 6 10.5 8 10 5.5
12 10 8 12 4 11
12 10 4 11 1 9
12 10 1 9 8.5 8.5
12 10 8.5 8.5 10.5 8
10 5.5 8 6 4.5 5.5
10 5.5 4.5 5.5 5.5 3.5
4 1.5 2 2 -1 1
2 2 2 5 -2 3
2 2 -2 3 -1 1
2 5 4.5 5.5 -1 6
2 5 -1 6 -2 3
4.5 5.5 8 6 -1 6
8 6 8.5 8.5 -1 6
8.5 8.5 1 9 -1 6
5
13 2
12 10
-1 6
2 2
10 5.5
```

This is the best stress example for the current code. It forces:

- multiple bridge events,
- repeated visits to bridge endpoints in the merged walk,

- nontrivial monotone face extraction,
- a guard set that is strictly smaller than the theoretical worst-case bound.

10 File-by-File Walkthrough

10.1 Main Driver

`src/main.cpp` is short but important because it defines the exact execution order:

1. parse one gallery,
2. count original vertices,
3. count holes,
4. merge holes,
5. triangulate the merged gallery,
6. compute guards from the triangulation result,
7. print triangles and guards.

The sequence is correct: both the triangle formula and the guard upper bound depend on the original gallery counts, so those numbers must be captured before hole merging changes the topology.

10.2 Parser Module

`src/parser.cpp` is responsible for converting coordinate input into the half-edge gallery representation. Its strongest contribution is the orientation normalization step. Without that invariant, the later sweep-line predicates would become unreliable.

10.3 Polygon Utility Module

`src/polygon_utils.cpp` is small, but it gives the traversal routines that keep the rest of the code clean. In particular, the hole merger depends on `getTopmostVertices` and the guard stage depends on `getTotalVertexCount`.

10.4 Hole Merger

`src/hole_merger.cpp` combines geometric ordering, angular indexing, component tracking, same-level helper locking, bridge target selection, and local pointer rewiring.

10.5 Triangulation Engine

`src/triangulation.cpp` is the densest algorithmic file. It contains:

- occurrence-level polygon construction,
- vertex classification,
- monotone decomposition,
- face extraction by circular embedding,

- monotone-face triangulation,
- final triangle filtering and printing support.

10.6 Guard Module

`src/guards.cpp` converts triangulation output into a graph problem and then back into geometric guard coordinates. The code is concise, but it correctly captures the theoretical idea: 3-color the triangulation graph and choose the smallest color class.

10.7 Build and Run Layer

`Makefile` compiles all source files in `src/` into `build/art_gallery`. `test.sh` builds the solver and runs it on the supplied input.

11 Conclusion

The program solves the art gallery problem for a polygon with holes by building a consistent geometric representation of the input, converting the gallery into a single merged boundary, triangulating that merged polygon, and then selecting a valid guard set from the triangulation. It first parses the outer boundary and holes into a half-edge structure. It then inserts bridges to connect each hole to a suitable wall so that the remaining stages can work on one boundary walk. The triangulation stage treats this merged walk as an ordered sequence of positions, which allows repeated bridge endpoints to be handled correctly, decomposes the polygon into monotone faces, recovers those faces from the diagonal graph, and triangulates them. Finally, the guard computation builds the adjacency graph induced by the triangulation, computes a three-coloring, and outputs the smallest color class as the guard set. The result is a complete end-to-end pipeline that takes the original gallery as input and returns both a valid triangulation and a guard placement on the original vertices.